Alright, let's build a clean, **NGINX + RBAC lab** where a user/service account can **manage only one specific Deployment** and nothing else.

I'll keep it **fully step-by-step with commands + YAML** so you can drop it straight into your training.

---

**Lab Goal**

- Namespace: rbac-nginx-demo
- Deployment: nginx-deploy
- ServiceAccount: nginx-operator
- Role: nginx-deployment-manager
- RBAC rule:
  ✅ Can *only* manage nginx-deploy (scale/update, view pods)
  ❌ Cannot create/delete other deployments in the namespace

---

**0. Prerequisites / Assumptions**

- You have a working Kubernetes cluster.
- kubectl is configured and talking to the cluster as an admin (cluster-admin / kubeconfig).
- You're in the right context:

kubectl config current-context

---

**1. Create a Separate Namespace**

*kubectl create namespace rbac-nginx-demo*

Verify:

*kubectl get ns rbac-nginx-demo*

---

**2. Create an NGINX Deployment in That Namespace**

You can use kubectl create or YAML. I'll show both; use whichever fits your lab style.

**Option A – CLI**

*kubectl create deployment nginx-deploy   --image=nginx  -n rbac-nginx-demo*

*kubectl get deploy -n rbac-nginx-demo*

*kubectl get pods -n rbac-nginx-demo*

**Option B – YAML (if you want to apply a file)**

**nginx-deploy.yaml**

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deploy
  namespace: rbac-nginx-demo
  labels:
    app: nginx-demo
spec:
  replicas: 1
  selector:
    matchLabels:
      app: nginx-demo
  template:
    metadata:
      labels:
        app: nginx-demo
    spec:
      containers:
        - name: nginx
          image: nginx:latest
          ports:
            - containerPort: 80
```

Apply:

*kubectl apply -f nginx-deploy.yaml*

*kubectl get deploy,pods -n rbac-nginx-demo*

---

**3. Create a Service Account for NGINX Operator**

This Service Account will represent our “limited user”.

**nginx-operator-sa.yaml**

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: nginx-operator
  namespace: rbac-nginx-demo
```

Apply:

```
kubectl apply -f nginx-operator-sa.yaml
kubectl get sa -n rbac-nginx-demo
```

You should see:

```
NAME           SECRETS   AGE
default        1         ...
nginx-operator 1         ...
```

---

**4. Create a Role That Can Only Manage *This* Deployment**

We’ll create a **Role** that:

- Can only **get/list/watch/update/patch** the Deployment named nginx-deploy
- Can **view pods** in the namespace (so they can see effect of scaling)
- **Cannot** touch other deployments or resources

Key point: we use resourceNames to restrict to **one specific Deployment**.

**nginx-deployment-role.yaml**

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: nginx-deployment-manager
```

```
  namespace: rbac-nginx-demo
rules:
  # Allow management of only this specific deployment
  - apiGroups: ["apps"]
    resources: ["deployments"]
    resourceNames: ["nginx-deploy"]
    verbs: ["get", "list", "watch", "update", "patch"]

  # Allow reading pods (to verify rollout)
  - apiGroups: [""]
    resources: ["pods"]
    verbs: ["get", "list", "watch"]
```

**Apply:**

```
kubectl apply -f nginx-deployment-role.yaml

kubectl get role -n rbac-nginx-demo
```

---

**5. Bind the Role to the Service Account**

Create a RoleBinding that ties nginx-operator → nginx-deployment-manager.

**nginx-deployment-rolebinding.yaml**

```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: nginx-deployment-manager-binding
  namespace: rbac-nginx-demo
subjects:
  - kind: ServiceAccount
    name: nginx-operator
    namespace: rbac-nginx-demo
roleRef:
  kind: Role
  name: nginx-deployment-manager
  apiGroup: rbac.authorization.k8s.io
```

**Apply:**

*kubectl apply -f nginx-deployment-rolebinding.yaml*

*kubectl get rolebinding -n rbac-nginx-demo*

---

**6. Test RBAC with kubectl auth can-i (Impersonation)**

Now we'll **simulate** actions as the Service Account using --as.

Note: This works if your cluster allows user impersonation (most labs with cluster-admin do).

**6.1. Check allowed actions on nginx-deploy**

*kubectl auth can-i get deployment/nginx-deploy -n rbac-nginx-demo --as=system:serviceaccount:rbac-nginx-demo:nginx-operator*

Should return:

***yes***

Try update:

*kubectl auth can-i update deployment/nginx-deploy -n rbac-nginx-demo --as=system:serviceaccount:rbac-nginx-demo:nginx-operator*

Should be:

***yes***

Check pods:

*kubectl auth can-i list pods -n rbac-nginx-demo --as=system:serviceaccount:rbac-nginx-demo:nginx-operator*

Should be:

***Yes***

### 6.2. Check actions that should be denied

Try to create a *new* deployment:

*kubectl auth can-i create deployments -n rbac-nginx-demo  --as=system:serviceaccount:rbac-nginx-demo:nginx-operator*

Expected:

***no***

Try to delete nginx-deploy:

***kubectl auth can-i delete deployment/nginx-deploy  -n rbac-nginx-demo  --as=system:serviceaccount:rbac-nginx-demo:nginx-operator***

Expected:

***no***

(optional) Create another deployment as admin:

*kubectl create deployment other-deploy  --image=nginx  -n rbac-nginx-demo*

Then test:

*kubectl auth can-i get deployment/other-deploy -n rbac-nginx-demo --as=system:serviceaccount:rbac-nginx-demo:nginx-operator*

Expected:

***no***

This clearly demonstrates **deployment-level RBAC**.

---

### 7. Actually Scale the Deployment as the Limited Account

If impersonation is allowed, you can simulate the nginx-operator scaling the deployment:

*kubectl scale deployment/nginx-deploy --replicas=3 -n rbac-nginx-demo --as=system:serviceaccount:rbac-nginx-demo:nginx-operator*

Check pods:

*kubectl get pods -n rbac-nginx-demo*

If you try to delete:

*kubectl delete deployment/nginx-deploy -n rbac-nginx-demo --as=system:serviceaccount:rbac-nginx-demo:nginx-operator*

You should get a **Forbidden** error – good for students to see.

---

**8. (Optional) Use the Service Account from Inside a Pod**

If you want to extend this into a **controller pattern** lab:

- Add serviceAccountName: nginx-operator under spec.template.spec of some "admin tool" Pod or Deployment.
- Any in-cluster application using the mounted token will then **only** have the limited permissions defined by nginx-deployment-manager.

Example patch to a Pod/Deployment:

```
spec:
  template:
    spec:
      serviceAccountName: nginx-operator
```

---

**9. Cleanup Commands (End of Lab)**

For a clean lab reset:

*kubectl delete namespace rbac-nginx-demo*

That removes everything in one shot.